MCPulse - MCP Server Monitoring & Performance Analytics Platform

A cloud-native observability platform for Model Context Protocol servers, deployed on AWS

Academic project - MCA Semester III, Cloud Computing

Abstract

Model Context Protocol (MCP) servers are becoming a standard interface for AI agents, but they are typically deployed as a black box: an agent either gets a tool response or it does not, and there is no record of how the server was behaving a minute ago. **MCPulse** is a monitoring and analytics platform for remotely reachable MCP servers. It actively probes each registered endpoint with the MCP `initialize` handshake, measures the real round-trip latency, persists every observation to a relational database, raises incidents after repeated failures, resolves them automatically on recovery, discovers the tool catalogue of each server, and presents the result as a live dashboard with an evidence-based diagnostics report.

The platform is deployed end to end on **Amazon Web Services** (EC2 for the application tier, RDS PostgreSQL for persistence, VPC and security groups for network isolation, CloudWatch for observability) and was verified with 1333 recorded checks, live tool discovery against a public MCP server and a complete incident lifecycle. The deployment is reproducible through a set of idempotent scripts and a documented resume/stop workflow for the AWS Academy Learner Lab.

---

1. Problem statement

MCP servers expose tools to AI agents, yet there is no standard way to answer:

- *Is the server up right now, and how fast is it?*
- *When did it start failing, and for how long?*
- *Which tools does it actually expose?*
- *Did the failure come from the network, from authentication, or from the server itself?*

Without this, an integration looks "broken" and debugging starts with `curl` loops and guesswork.

---

2. Objectives

#	Objective	Where it is realised
O1	Continuously measure availability and latency of registered MCP servers	Scheduler + MCP checker (§4.2)
O2	Persist every observation with a queryable relational model	PostgreSQL schema + migration engine (§4.1)
O3	Detect outages and manage the incident lifecycle automatically	Failure threshold + auto-resolve (§4.3)
O4	Discover and inventory the tools a server exposes	`tools/list` discovery (§4.4)
O5	Explain failures with evidence instead of guesses	Diagnostics report (§4.5)
O6	Deploy the platform on AWS with real isolation, encryption and observability	Cloud architecture (§5)
O7	Make the environment reproducible and cheap for a lab	Automation scripts (§6)

3. Scope

In scope

- Remotely reachable MCP servers over the Streamable HTTP transport.
- Five monitored providers: **GitHub, Google Drive, Brave Search, Cloudflare, Hugging Face**.
- A responsive web dashboard (React + Vite + Tailwind), an Express/TypeScript API, PostgreSQL persistence and a Docker Compose production stack.

Out of scope (deliberate)

- Local stdio-only servers and interception of AI client tool calls.
- Self-hosting or installing any third-party MCP server - MCPulse only *observes* endpoints that already exist.
- Per-provider credential management (would require a schema change and secret handling for OAuth/PAT material).

4. System design

4.1 Data model (PostgreSQL 16)

Table	Purpose
`mcp_servers`	Registered endpoints: name, URL, transport, interval, enabled flag
`monitoring_checks`	One row per probe: timestamp, `duration_ms`, outcome, HTTP status, sanitised error
`incidents`	Outage lifecycle: open/resolved, start/end timestamps, triggering check
`mcp_tools`	Discovered tool inventory: name, description, JSON Schema input
`tool_test_profiles`	Custom tool test inputs and expected validation rules
`tool_monitoring_checks`	Tool execution latency, outcome and validation result
`tool_incidents`	Tool-specific failures
`schema_migrations`	Applied migration history (3 migrations)

A transactional migration engine applies numbered SQL files, each in its own transaction, and records the applied filename - the history is preserved and re-running is a no-op.

4.2 The monitoring engine

- A `node-cron` scheduler evaluates every **10 seconds** and starts a check only when the server's configured interval (30-3600 s) is due.
- An in-memory lock per server prevents overlapping checks on a slow endpoint.
- The checker sends a real MCP JSON-RPC 2.0 `initialize` request with protocol version `2025-03-26`, tracking the `Mcp-Session-Id` header, and accepts both JSON and request-scoped **SSE** responses.
- Latency is measured as full round-trip time and stored in milliseconds.
- **SSRF protection** blocks loopback, private, carrier-grade-NAT, link-local and cloud metadata destinations, both literally and after DNS resolution, and it is applied *before every outbound request*.

4.3 Incident lifecycle

```
1 failure  ->  Degraded
3 consecutive failures  ->  Down  +  incident opened
first success afterwards ->  Healthy  +  incident auto-resolved
```

4.4 Tool discovery and testing

`tools/list` is sent to a chosen server, the returned catalogue is stored with its JSON Schema, and individual tools can be invoked (`tools/call`) with a measured duration and optional field-level validation rules.

4.5 Evidence-based diagnostics

`GET /api/analysis/:id` separates **observed facts** (measured checks, real latency, catalogue size, incident state) from **hypotheses** (for example *"the endpoint requires a bearer token"*) and ends with concrete **recommendations**, with the provider-specific reason for a 401 or 403 spelled out.

4.6 Frontend

React 19 + Vite + Tailwind v4: live status counts (Healthy / Degraded / Down / Unknown / Disabled), a Recharts latency view, the tool explorer with a manual test trigger, incident list, one-click CSV telemetry export (FR-12) and a dark / off-white theme.

---

5. Cloud architecture (AWS)

Service	Role	Live identifier
Amazon VPC (default)	Network isolation, 6 subnets over 6 AZs	`vpc-09e2f7d67772fdfea`
Amazon EC2 `t3.small`	nginx + Node API + scheduler, Docker Compose	`i-0059c1813dd3d1ada` - `54.161.184.121`
Amazon RDS PostgreSQL 16	Private persistence, 20 GB gp3	`mcpulse-db.cyinxjr3lgjz.us-east-1.rds.amaz`
Security groups	Web tier (80/443 public, 22 single address) and database tier (5432 from the web SG only)	`sg-085f461886a0676f7`, `sg-0d74bb805b76e8e0e`
CloudWatch Logs / Metrics	Application and host observability, 7-day retention	`/mcpulse/…`, namespace `CWAgent`
IAM	`LabRole` present but not attachable (`iam:PassRole` denied)	`LabInstanceProfile`
Amazon S3	Deliberately not used - the CSV export is produced from the database	-

Data flow

```
browser   80    nginx     Express API   TLS 5432    RDS PostgreSQL

                          HTTPS 443    the five monitored MCP providers
                            CloudWatch Logs (app + system)
                            CloudWatch Metrics (mem, disk)
```

Security posture

- The database has no public address and accepts connections only from the web security group; RDS rejects unencrypted connections outright.
- The API port is published on `127.0.0.1` only - nginx is the single public entry point.
- Every state-changing API call must carry the administrative key.
- The SSRF guard rejects metadata and private targets before any request.
- No secret is stored in the repository; the database password, the SSH key and the AWS credentials live outside the project tree.

---

6. Reproducibility and lab economics

Five idempotent scripts make the environment reproducible and cheap:

Script	Purpose
`audit.sh`	Read-only readiness inspection (identity, VPC, resources, permissions)
`deploy.sh`	Create or update the infrastructure and the application
`resume.sh`	Restart a stopped deployment after a lab session (database first, then the instance)
`stop.sh`	Stop the resources to stop the credit burn - deletes nothing
`allowlist.sh`	Refresh the administrator address after a network change

Cost: ≈ **$0.18 per four-hour lab session** (EC2 $0.021/h, RDS $0.017/h, 20 GB gp3, public IPv4) - no load balancer, no NAT gateway, no Multi-AZ, no S3.

---

7. Verification results

Requirement	Verification	Result
Continuous availability and latency monitoring	1333 rows in `monitoring_checks`, written every provider interval	OK
Relational persistence and migration history	3 migrations applied; `schema_migrations` records them	OK
Outage detection and incident lifecycle	3 consecutive failures -> `Down` + open incident; recovery -> `Healthy` + auto-resolve	OK
Tool discovery	4 live tools from the Hugging Face MCP server (`hf_fs`, `hf_whoami`, `hub_repo_details`, `hub_repo_search`)	OK
Telemetry CSV export (FR-12)	364 rows generated on demand	OK
Health and readiness endpoints	`/health` (liveness) and `/health/ready` (verifies the database)	OK
Database isolation	`PubliclyAccessible = false`; unencrypted connections refused by RDS	OK
SSRF protection	Metadata, private and loopback targets rejected with 400	OK
Administrative authorization	Write without the key -> 401; with the key -> success	OK
Authentication-gated providers	GitHub 401, Cloudflare 401, Brave 403 recorded with the exact status	OK

---

8. Challenges encountered and how they were solved

Challenge	Resolution
The container build failed: npm workspaces keep the lockfile at the repository root	Build context moved to the repository root with a workspace-aware install
The dashboard called `localhost:3001` in the visitor's browser	Same-origin API calls plus a Vite dev proxy
`pg` silently discards a verified TLS configuration when the URL contains `sslmode=require`	The URL is stripped of every `ssl*` parameter and the CA bundle is enforced explicitly
The SSRF guard existed but was never called	Wired into the registry routes and into every outbound request
macOS AppleDouble files (`._*.sql`) were executed as SQL on the database	The runner ignores hidden files and the transfer strips extended attributes
A multi-statement migration aborted the database transaction and masked the real error	The runner executes statement by statement and reports the exact statement
The lab policy denies `rds:CreateDBInstance` *according to the IAM simulation*, but the real API allowed it	Permissions proven by calling the API, not by the simulation
The campus network changes the public address repeatedly	Address allowlist replaced by an administrative key; one script refreshes SSH

---

9. Limitations and future work

Limitations

1. No per-provider credentials - three of five providers require a token, and MCPulse correctly reports them as failing.
2. No instance profile in this account, so the CloudWatch agent uses temporary credentials and delivery stops with the lab session.
3. CloudWatch log events and metric datapoints do not arrive in this account (the agent runs and creates the groups; the API path itself is verified healthy).
4. The administrative key is compiled into the dashboard bundle, so write operations are protected against casual use rather than against a determined attacker.
5. The project is not under version control, so the deployment transfers sources with `tar` over SSH.

Future work

- OAuth 2.0 / PAT support with encrypted token storage and per-provider headers.
- SLO and error-budget reporting with burn-rate alerts on CloudWatch.
- Distributed tracing with AWS X-Ray and correlation IDs in the check records.
- Automated remediation (restart, failover) driven by incident state.
- A Terraform module so the environment is reproducible outside the lab.

---

10. Conclusion

MCPulse demonstrates the complete path from a monitoring requirement to a running cloud platform: a relational model of the domain, a protocol-aware checker, an automated incident lifecycle, a live dashboard, and a deployment that uses the right AWS service for each concern - compute on EC2, managed persistence on RDS, isolation in the VPC, least-privilege networking with security groups, and observability through CloudWatch - while remaining reproducible and affordable inside a strict lab budget.

References

1. Model Context Protocol specification - protocol version `2025-03-26`, Streamable HTTP transport, `initialize` / `tools/list` / `tools/call`.
2. AWS Academy Learner Lab environment, account `462854211377`, `us-east-1`.
3. AWS documentation: EC2, Amazon RDS for PostgreSQL, Amazon VPC, security groups, Amazon CloudWatch (Logs, Metrics, unified agent).
4. Project documents: `docs/PROJECT_STATUS_AND_AWS_GUIDE.md`, `docs/AWS_ARCHITECTURE_AND_DATA_FLOW.md`, `docs/DEPLOYMENT_STATUS.md`, `docs/phase-0-decisions.md`, `docs/DEMONSTRATION_GUIDE.md`.